

Министерство образования и науки Российской Федерации
Московский физико-технический институт (государственный университет)

Физтех-школа радиотехники и компьютерных технологий
Кафедра микропроцессорных технологий в интеллектуальных системах управления
YADRO

Выпускная квалификационная работа бакалавра

Реализация в отладчике GDB поддержки вызова функций с векторным ABI для архитектуры RISC-V

Автор:

Студент Б01-110 группы
Радькин Кирилл Алексеевич

Научный руководитель:

Владимиров Константин Игоревич



Москва 2025

Аннотация

Реализация в отладчике GDB поддержки вызова функций с векторным ABI для архитектуры RISC-V

Радькин Кирилл Алексеевич

;

Содержание

1	Введение	4
2	Постановка задачи	5
3	Обзор архитектуры RISC-V	6
3.1	Общая информация	6
3.2	Векторное расширение	7
3.2.1	Регистр <code>vtype</code>	7
3.2.2	Регистр <code>v1</code>	9
3.2.3	Регистр <code>vlenb</code>	9
4	Отладчик GDB	10
4.1	Основная информация	10
4.2	Основные возможности	10
4.3	Отладочная информация: DWARF	11
4.3.1	DIE	11
5	Описание практической работы	15

1 Введение

;

2 Постановка задачи

Цель работы: реализовать в отладчике GDB для архитектуры RISC-V поддержку вызова функций, использующих векторный ABI. Для выполнения данной цели были поставлены следующие задачи:

1. Изучить генерацию отладочной информации для RISC-V компиляторами GCC и LLVM
2. Реализовать RISC-V Vector Calling Convention в отладчике GDB
3. Поддержать определение векторных типов и работу с ними в GDB

3 Обзор архитектуры RISC-V

3.1 Общая информация

RISC-V – это архитектура команд (ISA), основанная на принципах сокращённого набора команд (RISC). Она была разработана в Калифорнийском университете в Беркли в 2010 году и предназначена как для обучения, так и для промышленного использования.

Одной из ключевых особенностей архитектуры является ее открытость. Спецификации RISC-V находятся в открытом доступе. Архитектура свободна от лицензионных отчислений, что позволяет использовать ее без юридических или финансовых ограничений.

Еще одним важным свойством архитектуры является ее модульность. Архитектура состоит из базового набора целочисленных инструкций, который должен присутствовать в любой реализации, и из дополнительных расширений, которые можно включать по необходимости. Существует 4 варианта базового набора целочисленных инструкций: RV32I и RV64I, которые описывают 32-разрядные или 64-разрядные адресные (далее разрядность будем называть XLEN, в терминах спецификации RISC-V) пространства, и их подмножества, RV32E и RV64E, которые были добавлены для поддержки небольших микроконтроллеров и обладающие вдвое меньшим набором целочисленных регистров. Дополнительные расширения можно разделить на две группы: стандартные и пользовательские. Среди стандартных дополнительных расширений можно выделить следующие:

- M – добавляет целочисленное умножение/деление
- A – добавляет атомарные операции с памятью
- F – добавляет регистры с плавающей точкой и соответствующие операции
- D – добавляет регистры с плавающей точкой двойной точности и соответствующие операции
- C – набор сжатых инструкций (с уменьшенной длиной)
- V – векторные регистры и соответствующие операции
- B – битовые операции

- **Zicsr** – инструкции для работы с регистрами состояния и статуса ядра (Control and Status Registers, CSR)
- **Zifencei** – инструкции для синхронизации потока инструкций и потока данных

Кроме того, существует аббревиатура G для обозначения базового набора расширений `IMAFD_Zicsr_Zifencei`.

Архитектура поддерживается организацией RISC-V International, которая разрабатывает и утверждает стандартные расширения.

3.2 Векторное расширение

Векторное расширение (RISC-V Vector Extension, RVV) описывает набор инструкций и регистров, предназначенных для эффективной параллельной обработки данных.

Каждое ядро, поддерживающее данное расширение, определяет два значения:

1. **ELEN** – максимальный размер векторного элемента (в битах), который может быть использован или создан векторной операцией. $ELEN \geq 8$ и должно быть степенью 2.
2. **VLEN** – количество бит в одном векторном регистре. $2^{16} \geq VLEN \geq ELEN$, и должно быть степенью 2.

Векторное расширение добавляет 32 векторных регистра (`v0-v31`) и 7 непривилегированных CSR (`vstart`, `vxstat`, `vxrm`, `vcsr`, `vtype`, `vl`, `vlenb`).

3.2.1 Регистр `vtype`

Регистр `vtype` является read-only CSR (т. е. регистр, доступный только для чтения) и может быть обновлен только с помощью специальных инструкций `vset{i}vl{i}`.

Данный регистр хранит в себе следующую информацию:

- Интерпретацию значений векторных регистров
- Способ группировки векторных регистров

- Способ обработки маскированных элементов
- Способ обработки элементов, длина которых превышает текущую длину вектора



Рис. 1: Поля регистра vtype

- **vill** – флаг, указывающий на нелегальное значение
- **vma** – если 0, то немаскированные элементы сохраняют свои значения после операций, если 1, то они могут как сохранить, так и быть переписаны единицами
- **vta** – аналогично **vma**, только для элементов, которые находятся за текущей длиной вектора
- **vsew** – текущее значение размера одного элемента (Selected Element Width, SEW)

vsew[2:0]			SEW
0	0	0	8
0	0	1	16
0	1	0	32
0	1	1	64
1	X	X	Reserved

Таблица 1: Соответствие между vsew[2:0] и SEW

Значение SEW – это размер в битах одного скалярного значения внутри векторного регистра. Соответственно, количество элементов в одном векторном регистре вычисляется как $VLEN/SEW$. В таблице 2 приведен пример для $VLEN = 128$ бит.

SEW	Количество элементов
8	16
16	8
32	4
64	2

Таблица 2: Пример для $VLEN = 128$ бит

- `vlmul` – текущее значение размера группы векторных регистров (Length Multiplier, LMUL)

Векторные регистры могут быть сгруппированы, чтобы одна векторная инструкция могла оперировать сразу с несколькими векторными регистрами. Это повышает эффективность обработки данных, позволяя работать сразу с несколькими регистрами (т. е. с векторами увеличенной длины). Значение LMUL представляет собой количество векторных регистров, сгруппированных вместе.

Кроме того, LMUL может быть дробным значением. Дробное LMUL означает, что используется только часть бит векторного регистра.

Таким образом, мы можем вычислить максимальное количество скалярных элементов, которыми может оперировать одна векторная инструкция с текущими SEW и LMUL, как $VLMAX = LMUL * VLEN / SEW$.

3.2.2 Регистр `vl`

Регистр `vl` является read-only CSR и может быть обновлен только с помощью специальных инструкций `vset{i}vl{i}`.

Данный регистр хранит в себе беззнаковое целочисленное значение, определяющее, с каким количеством скалярных элементов будет взаимодействовать векторная инструкция.

3.2.3 Регистр `vlenb`

Регистр `vlenb` является read-only CSR, который хранит в себе длину одного векторного регистра в байтах, т. е. $VLEN/8$.

4 Отладчик GDB

4.1 Основная информация

GDB (GNU Project debugger) – отладчик, программа, позволяющая анализировать и управлять исполнением другой программы для нахождения ошибок в коде. GDB поддерживает отладку программ, написанных на следующих языках:

- Ada
- Assembly
- C/C++
- D
- Fortran
- Go
- Objective-C
- OpenCL
- Modula-2
- Pascal
- Rust

4.2 Основные возможности

GDB позволяет запускать программы под своим управлением, анализировать и управлять их исполнением.

Одна из основных возможностей это остановка исполнения программы в некоторой точке. GDB позволяет устанавливать специальные метки в программе – breakpoint-ы (точки останова), которая указывает, в каком месте следует остановить выполнение программы. Breakpoint может быть привязан как к строке в коде, так и к определенному адресу. Во втором случае исполнение программы будет остановлено перед исполнением инструкции, расположенной по указанному адресу.

После остановки программы, GDB позволяет выполнять множество различных операций, полезных для поиска ошибок в коде.

Например, GDB позволяет просматривать значение переменных и изменять их. Для низкоуровневой отладки существует аналогичная возможность отслеживания и редактирования значений определенных регистров.

Следующей важной возможностью GDB является просмотр информации о текущем стековом фрейме и стеке вызова функций.

Также, весьма полезной функцией является вызов отдельных функций программы.

4.3 Отладочная информация: DWARF

Отладочная информация – данные, которые компилятор вставляет в исполняемые файлы. Эти данные позволяют GDB понять, какие существуют переменные, функции, их местонахождение, как соотносятся между собой машинные инструкции и исходный код и т. д.

DWARF – это современный формат отладочной информации, который используется в компиляторах GCC и LLVM и поддерживается GDB.

Стандарт DWARF разработан так, чтобы быть применимым к множеству существующих языков, а также иметь возможности к расширению для будущих языков.

4.3.1 DIE

Запись отладочной информации (Debugging Information Entry, DIE) – это базовая единица данных в формате DWARF. Записи отладочной информации хранятся в секции `.debug_info` и/или в секции `.debug_info.dwo` объектного файла.

DWARF использует наборы таких записей, чтобы описать структуру программы. Каждая запись состоит из тега (tag), определяющего тип описываемой сущности (например функция, переменная или название типа), и атрибутов, задающих конкретные свойства.

Рассмотрим генерацию DWARF информации на конкретном примере. Здесь и далее для компиляции я буду использовать LLVM Clang 19.1.0 (если не указано иное) для архитектуры RISC-V, а для наглядного отображения DWARF информации – инструмент `llvm-dwarfdump`. Данный инструмент

сканирует DWARF секции в объектных файлах и выводит содержимое в удобочитаемом виде.

Итак, возьмем простую функцию с Листинга 1 и посмотрим, какую DWARF информацию генерирует для нее компилятор.

```
int foo (char a, int b[3], float *c) {  
    return 42;  
}
```

Листинг 1: Пример простейшей функции

Для данной функции будет сгенерирована следующая DWARF информация.

```
0x00000091:    DW_TAG_subprogram  
                DW_AT_low_pc      (0x00000000000000654)  
                DW_AT_high_pc     (0x00000000000000674)  
                DW_AT_frame_base  (DW_OP_reg8 X8)  
                DW_AT_name        ("foo")  
                DW_AT_decl_file   ("/app/example.c")  
                DW_AT_decl_line   (1)  
                DW_AT_prototyped   (true)  
                DW_AT_type        (0x000000d1 "int")  
                DW_AT_external    (true)  
  
0x000000a0:    DW_TAG_formal_parameter  
                DW_AT_location     (DW_OP_fbreg -17)  
                DW_AT_name         ("a")  
                DW_AT_decl_file    ("/app/example.c")  
                DW_AT_decl_line    (1)  
                DW_AT_type         (0x000000d5 "char")  
  
0x000000ab:    DW_TAG_formal_parameter  
                DW_AT_location     (DW_OP_fbreg -32)  
                DW_AT_name         ("b")  
                DW_AT_decl_file    ("/app/example.c")  
                DW_AT_decl_line    (1)  
                DW_AT_type         (0x000000d9 "int *")
```

```
0x000000b6:    DW_TAG_formal_parameter
                DW_AT_location    (DW_OP_fbreg -40)
                DW_AT_name      ("c")
                DW_AT_decl_file  ("/app/example.c")
                DW_AT_decl_line  (1)
                DW_AT_type       (0x000000de "float *")

...

0x000000d1:    DW_TAG_base_type
                DW_AT_name      ("int")
                DW_AT_encoding   (DW_ATE_signed)
                DW_AT_byte_size  (0x04)

0x000000d5:    DW_TAG_base_type
                DW_AT_name      ("char")
                DW_AT_encoding   (DW_ATE_unsigned_char)
                DW_AT_byte_size  (0x01)

0x000000d9:    DW_TAG_pointer_type
                DW_AT_type       (0x000000d1 "int")

0x000000de:    DW_TAG_pointer_type
                DW_AT_type       (0x000000e3 "float")

0x000000e3:    DW_TAG_base_type
                DW_AT_name      ("float")
                DW_AT_encoding   (DW_ATE_float)
                DW_AT_byte_size  (0x04)
```

Листинг 2: Часть сгенерированной DWARF-информации, соответствующая функции foo

На Листинге 2 можно увидеть тег `DW_TAG_subprogram`, указывающий на то, что данная запись описывает некоторую функцию. По тегу `DW_AT_name`

мы можем определить, что это функция с именем `foo`.

Атрибуты `DW_AT_low_pc` и `DW_AT_high_pc` описывают диапазон адресов в машинном коде, в котором располагается данная функция.

Содержимое `DW_AT_frame_base` описывает значение базового адреса фрейма данной функции. Для данной функции адрес ее фрейма будет располагаться на регистре `x8`.

Для описания типа возвращаемого функцией значения, используется атрибут `DW_AT_type`

5 Описание практической работы

Список литературы

- [1] *Mott-Smith, H.* The theory of collectors in gaseous discharges / H. Mott-Smith, I. Langmuir // *Phys. Rev.* — 1926. — Vol. 28.